

REPORTE TÉCNICO DE PROYECTO

APLICACIÓN MÓVIL: CONVERSIONOR DE UNIDADES

Desarrollo Android Nativo con Jetpack Compose, Arquitectura MVVM y StateFlow Reactivo

Estudiante:	Edgar Solis Hernandez
Carrera / Programa:	Ingeniería en Tecnologías de la Información e Innovación Digital
Institución:	Universidad Politécnica de Chiapas (UPChiapas)
Grado y Grupo:	4° Cuatrimestre — Grupo 4B
Asignatura:	Programación y Desarrollo de Aplicaciones Móviles
Repositorio Oficial:	https://github.com/lolxde121/Conversor-Unidades
Fecha de Elaboración:	Octubre de 2026
Entorno Tecnológico:	Android SDK 37 (minSdk 30) Kotlin 2.0+ Material Design 3

SÍNTESIS DEL PROYECTO

El presente informe documenta en detalle el diseño, arquitectura, formulación matemática y desarrollo de la aplicación móvil **Conversionor de Unidades**, desarrollada por el estudiante **Edgar Solis Hernandez** del **Grupo 4B** en la **Universidad Politécnica de Chiapas**. La aplicación implementa el paradigma declarativo moderno en Android mediante **Jetpack Compose** y **Material 3**, estructurado bajo el patrón arquitectónico **MVVM (Model-View-ViewModel)** con sincronización bidireccional reactiva impulsada por **StateFlow** y corrutinas de Kotlin. Se incluye el enlace al código fuente abierto en GitHub: <https://github.com/lolxde121/Conversor-Unidades>.

1. Introducción y Justificación del Proyecto

En la ingeniería moderna y en las actividades cotidianas, la conversión entre diferentes sistemas de medida (Sistema Internacional y Sistema Imperial) es una tarea fundamental que exige rapidez, precisión y una interfaz libre de fricción. Muchas herramientas móviles convencionales obligan al usuario a presionar botones manuales de 'Calcular' o reiniciar campos al alternar unidades de origen y destino. El proyecto **Conversor de Unidades** desarrollado por **Edgar Solis Hernandez** para la carrera de **Ingeniería en Tecnologías de la Información e Innovación Digital** en la **Universidad Politécnica de Chiapas** supera estas limitaciones implementando una experiencia **100% reactiva e instantánea**: cualquier cambio numérico o de unidad actualiza el resultado de forma transparente y en tiempo real.

1.1. Objetivos del Proyecto

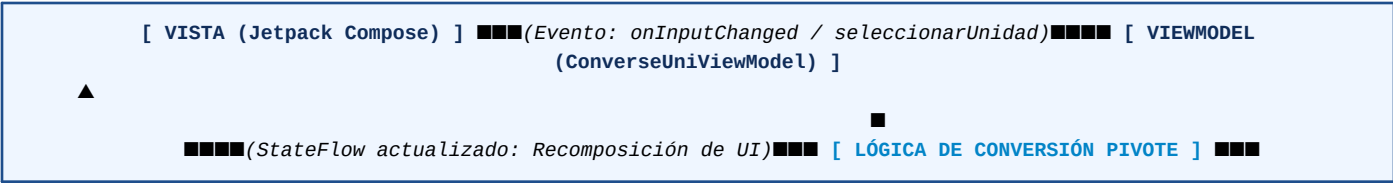
Objetivo General: Construir una aplicación móvil nativa en Android para la conversión precisa y bidireccional entre unidades de longitud (Centímetros, Metros, Kilómetros y Pulgadas), aplicando UI declarativa con Jetpack Compose y el patrón MVVM reactivo.

- **Modularidad:** Separar la presentación visual de la lógica de negocio mediante componentes desacoplados.
- **Reactividad:** Utilizar MutableStateFlow y StateFlow para propagación inmediata de estados.
- **Eficiencia Algorítmica:** Adoptar el metro como unidad pivote reduciendo la complejidad a O(N).
- **Ergonomía UI/UX:** Crear una interfaz estilizada con superficies elevadas, selectores badge y contraste sobrio.

2. Arquitectura de Software y Patrón MVVM

El proyecto implementa la arquitectura recomendada por Google: **Model-View-ViewModel (MVVM)**. Este patrón aísla el ciclo de vida de la actividad de los datos y asegura un código altamente testeable y mantenible.

Capa	Componentes	Responsabilidad Principal
Vista (UI)	ConverseVMPage.kt TopBarTitle.kt TitleUnit.kt	Renderizado declarativo con Jetpack Compose y Material 3. Captura eventos de usuario y recolecta estados mediante collectAsStateWithLifecycle().
ViewModel	ConverseUniViewModel.kt	Gestiona el estado reactivo mediante StateFlow. Procesa inputs de usuario, apertura/cierre de menús y coordina los recálculos.
Modelo / Lógica	Funciones convertir() y formatearNumero()	Lógica pura de normalización matemática, factores escalares respecto al metro y formateo numérico localizado con 2 decimales.



3. Modelo Matemático y Lógica de Conversión

Si se programaran conversiones directas entre cada par posible de unidades (ej. cm a m, cm a in, in a km, etc.), el número de combinaciones crecería de forma cuadrática: $K = N \times (N - 1)$. Para 4 unidades se requerirían 12 casos condicionales, y para 10 unidades se requerirían 90 ramas de código, multiplicando el riesgo de errores aritméticos.

La técnica de la Unidad Pivote (Metro): El software implementa el **Metro (m)** como unidad canónica de referencia. Cualquier magnitud de entrada se transforma primero a metros multiplicándola por su factor de escala, y posteriormente se divide entre el factor de la unidad destino. Esto reduce la complejidad a **O(N)**, logrando una arquitectura limpia y extensible.

Unidad de Longitud	Símbolo	Factor a Metros (Escalar k)	Ecuación Canónica
Centímetro	cm	0.01	metros = valor × 0.01
Metro (Pivote Base)	m	1.0	metros = valor × 1.0
Kilómetro	km	1000.0	metros = valor × 1000.0
Pulgada (Imperial)	in	0.0254	metros = valor × 0.0254

3.1. Algoritmo Bidireccional y Formateo Inteligente

Para evitar bucles de actualización infinita, las funciones `recalcularDesdeInput1` y `recalcularDesdeInput2` operan de forma asimétrica sobre el campo opuesto. Además, la función `formatearNumero` detecta si el resultado carece de parte decimal mediante `valor % 1.0f == 0.0f` (mostrando enteros limpios como '100' en vez de '100.00'), o aplica formato con dos decimales precisos (`Locale.US, "%.2f"`) en caso de números fraccionarios.

4. Especificaciones Técnicas y Stack Tecnológico

Componente	Versión / Especificación	Justificación y Rol en el Proyecto
Lenguaje	Kotlin 2.0+ (JVM 11)	Código conciso, seguro ante nulos, con soporte de primera clase para corrutinas y sintaxis reactiva.
UI Toolkit	Jetpack Compose (BOM)	Paradigma declarativo moderno que reemplaza XML, reduciendo boilerplate y optimizando recomposición.
Sistema Visual	Material Design 3 (M3)	Estándar actual de Google con elevación de superficies, bordes suaves y contraste refinado.
Gestión de Estado	StateFlow & ViewModel	lifecycle-viewmodel-compose con collectAsStateWithLifecycle para consumo seguro del ciclo de vida.
Android SDK	minSdk: 30 targetSdk: 37 compileSdk: 37	Compatibilidad garantizada desde Android 11 (R) hasta la versión más reciente (Android 15+).
Build System	Gradle con Kotlin DSL (*.kts)	Scripts tipados con verificación de dependencias y autocompletado en compilación.

5. Diseño de Interfaz de Usuario y Componentes (UI/UX)

La interfaz destaca por un diseño estilizado, ergonómico y minimalista compuesto por los siguientes módulos:

- **TopBarTitle.kt:** Barra superior cilíndrica (RoundedCornerShape de 200.dp) con fondo gris claro (#D9D9D9), sombra de 4dp y tipografía centrada de 27.sp.
- **TitleUnit.kt:** Insignia táctil con fondo negro, bordes redondeados (9.dp) y texto blanco nítido que despliega el menú contextual al pulsarse.
- **DropdownMenu Dinámico:** Menú emergente con la lista de magnitudes (Centímetros, Metros, Kilómetros, Pulgadas) para selección ágil.
- **Surface de Entrada Numérica:** Contenedores de 314x119 dp con esquinas redondeadas de 45.dp, marco BorderStroke de 16.dp en color LightGray y sombra táctil.
- **BasicTextField Personalizado:** Campo configurado con KeyboardOptions(keyboardType = KeyboardType.Number), texto blanco centrado de 27.sp.
- **Ícono Central de Intercambio:** Recurso gráfico drawable (intercambiar.png) ubicado entre ambos campos, comunicando visualmente la sincronía bidireccional.

6. Análisis de Código Fuente y Buenas Prácticas

A continuación se presenta el núcleo reactivo implementado en ConverseUniViewModel.kt:

```
// Estados reactivos encapsulados (MutableStateFlow privado / StateFlow público)
private val _unidad1 = MutableStateFlow("Centímetros")
val unidad1: StateFlow<String> = _unidad1.asStateFlow()

private val _inputText1 = MutableStateFlow("100")
val inputText1: StateFlow<String> = _inputText1.asStateFlow()

// Lógica de conversión normalizada mediante unidad pivote (Metro)
private fun convertir(valor: Float, deUnidad: String, aUnidad: String): Float {
    val metros = when (deUnidad) {
        "Centímetros" -> valor * 0.01f
        "Metros"       -> valor * 1.0f
        "Kilómetros"   -> valor * 1000.0f
        "Pulgadas"     -> valor * 0.0254f
        else           -> valor
    }
    return when (aUnidad) {
        "Centímetros" -> metros / 0.01f
        "Metros"       -> metros / 1.0f
        "Kilómetros"   -> metros / 1000.0f
        "Pulgadas"     -> metros / 0.0254f
        else           -> metros
    }
}

private fun recalcularDesdeInput1(texto: String) {
    val valor = texto.toFloatOrNull() ?: run { _inputText2.value = ""; return }
    val resultado = convertir(valor, _unidad1.value, _unidad2.value)
    _inputText2.value = formatearNumero(resultado)
}
```

Puntos de Ingeniería Relevantes:

1. **Encapsulamiento del Estado:** `_inputText1` y `_unidad1` son mutables privados, previniendo alteraciones no autorizadas fuera del ViewModel.
2. **Resiliencia con `toFloatOrNull()`:** Si el usuario borra la entrada o introduce caracteres especiales, el sistema limpia el campo complementario de forma segura en lugar de lanzar una excepción `NumberFormatException`.

7. Control de Versiones y Repositorio en GitHub

El desarrollo del proyecto fue gestionado rigurosamente mediante Git y alojado públicamente en GitHub: <https://github.com/lolxde121/Conversor-Unidades>. El historial evidencia una evolución incremental y estructurada:

Hash	Mensaje del Commit	Fase del Desarrollo
433bf97	primer:avance	Configuración inicial Gradle y estructura del proyecto
4665ae4	componente: topBar Personalizado	Diseño del componente TopBarTitle modular
2cb4369	primer version de componentes	Creación inicial de componentes Compose
da5526f	primer version de componentes	Refinamiento de parámetros en botones y campos
5db4dba	contenido inicial	Vinculación de la pantalla principal en MainActivity
8ff1573	campos de texto, menu desplegable	Integración de BasicTextField y DropdownMenu
f552762	primera version conversion	Lógica matemática y sincronización con ViewModel
684ac12	fix: ajuste en campo ahora se vea mejor la unidad dentro del campo negro	Ajuste visual y padding de texto en TitleUnit

8. Conclusiones y Trabajo Futuro

Conclusiones del Proyecto:

- **Eficiencia con Jetpack Compose:** La eliminación del XML redujo notablemente las líneas de código y facilitó un diseño reactivo y adaptativo.
- **Robustez del Patrón MVVM:** Concentrar la lógica y los estados en el ViewModel garantiza que la interfaz siempre refleje el estado real de la aplicación.
- **Escalabilidad:** La metodología de unidad pivote (metro) permite incorporar nuevas unidades de longitud con una sola línea de código adicional.
- **Desarrollo Académico:** El proyecto consolida las competencias de programación móvil del alumno **Edgar Solis Hernandez** en la **UPChiapas**.

Líneas de Trabajo Futuro:

- **Nuevas Categorías:** Incorporar pestañas para magnitudes de Masa (kg, lb), Temperatura (°C, °F, K) y Velocidad (km/h, mph).
- **Persistencia Local:** Integrar Room Database para almacenar el historial de conversiones recientes.
- **Internacionalización:** Soporte multilenguaje (inglés/español) y temas claros/oscuros adaptativos.

9. Referencias y Enlaces Oficiales

- **Repositorio GitHub del Proyecto:** <https://github.com/lolxde121/Conversor-Unidades> — Código fuente, commits e historial de versiones por Edgar Solis.
- **Guía de Arquitectura de Apps Android:** <https://developer.android.com/topic/architecture> — Patrones y recomendaciones oficiales de Google Developers.
- **Documentación de Jetpack Compose:** <https://developer.android.com/jetpack/compose> — Guías oficiales del toolkit moderno de interfaz declarativa.
- **Kotlin Coroutines y StateFlow:** <https://kotlinlang.org/docs/flow.html> — Documentación oficial de Kotlin para flujos reactivos asíncronos.